Diplomatura en Desarrollo de Software FinTech: IA y Microservicios

Clase 14

Actividad Práctica: API Fintech

Introducción

En el contexto actual del desarrollo de software, la construcción de APIs REST se ha convertido en una práctica fundamental para la comunicación entre **sistemas y la gestión eficiente de datos**. El objetivo principal es diseñar una solución escalable que permita modelar entidades del dominio, persistir información en una base de datos relacional como MySQL y exponer funcionalidades a través de endpoints REST.

En este trabajo se propone a los estudiantes desarrollar una API orientada a la gestión de cuentas y usuarios, aplicando conceptos trabajados durante las clases sincrónicas como Programación Orientada a Objetos (POO) y buenas prácticas.

Objetivo

- Desarrollar una API REST en Java (Spring Boot) que simule un sistema básico fintech, integrando:
    - Programación orientada a objetos
    - Persistencia en base de datos (MySQL)
    - Testing unitario
    - Documentación con Swagger

- Incorporar buenas prácticas en la estructuración de aplicaciones basadas en APIs REST.
- Comprender el proceso de diseño de una solución backend, desde el modelado del dominio hasta la exposición de servicios.
- Comprender la importancia del testing en el aseguramiento de la calidad del software.

Consigna

Construir una API que gestione **cuentas y usuarios**.

Requerimientos mínimos:

1. Modelo de dominio (POO)

Crear al menos:

- Cuenta
- Usuario

Aplicar:

- Herencia
- Interfaces o clases abstractas

2. API REST (Spring)

Endpoints mínimos:

- GET /cuentas
- POST /cuentas
- GET /cuentas/{id}

Bonus:

- DELETE /cuentas/{id}
- PUT /cuentas/{id}

3. Base de datos (MySQL)

- Persistir cuentas y usuarios
- Relación entre entidades
- Uso de repositorios (JPA)

4. Testing

Unitario

- Servicios
- Lógica de negocio

5. Manejo de errores

- try/catch
- Excepciones custom. Ejemplo:
    - Cuenta no encontrada

6. Documentación

- Swagger/OpenAPI
- Endpoints documentados

Entregable:

Un repositorio en Git que incluya:

- Código fuente completo
- README con instrucciones de ejecución
- Colección de endpoints (Postman o similar)
- Tests implementados

Tiempo de entrega: 21/10/2026

Criterios de evaluación:

- **Comprensión del problema y diseño de la solución**
  Claridad en el modelado del dominio y coherencia entre los requerimientos y la arquitectura propuesta.
- **Calidad de implementación**
  Uso adecuado de buenas prácticas de desarrollo en Spring Framework (organización por capas, legibilidad, modularidad, uso correcto de anotaciones). Claridad en la implementación y comprensión del proceso realizado.
- **Testing**
  Presencia y calidad de pruebas unitarias, validando lógica de negocio y servicios.
- **Documentación**
  Nivel de detalle y claridad en la documentación de la API (Swagger/OpenAPI) y del proyecto en general.
- **Autonomía y proceso de trabajo**
  Capacidad para investigar, tomar decisiones técnicas y resolver problemas de manera independiente.